KU LEUVEN

FACULTEIT INDUSTRIELE
INGENIEURSWETENSCHAPPEN

TECHNOLOGIECAMPUS GENT

Geavanceerde Computerarchitectuur

Report 2
MEI-ecfs

Charlotte BERT
Sander HAUSTRAETE

Academiejaar 2025 - 2026

1 Context

Het doel van dit labo is het implementeren van max detection op een 1D-array met drie methoden:

- Een for-lus op de CPU
- Een GPU-kernel met atomaire operaties
- Een GPU-kernel met reductie

Daarnaast wordt de uitvoeringstijd gemeten voor verschillende arraylengtes om de afhankelijkheid van de rekentijd van de inputgrootte te analyseren.

2 Methodologie

We begonnen met het schrijven van de code voor een implementatie met één block. Eerst schreven we de CPU-versie, daarna de GPU-versie met atomaire operaties en ten slotte de GPU-versie met reductie. Nadat alles correct werkte voor éń block hebben we de code uitgebreid naar meerdere blocks, afhankelijk van de grootte van de array.

Bij de GPU-reductie stuitten we echter op een probleem: thread-synchronisatie werkt enkel binnen één block en niet tussen verschillende blocks. Om dit op te lossen hebben we onze code aangepast. We gebruiken nu shared memory om per block het maximum te bepalen. Van deze lijst van gedeeltelijke maxima wordt dan nog eens het maximum bepaald. (zie codefragment 1)

3 Observatie/ resultaat

Is het resultaat correct?
Aangezien de array wordt opgebouwd via een for-lus die na elke iteratie een groter getal aan de array toevoegt, is het juiste maximum op voorhand gekend, namelijk $N - 1$. Na uitvoering van elke methode wordt gecontroleerd of het berekende maximum overeenkomt met dit correcte maximum, zoals te zien is in figuur 1.

Hoe neemt de tijd toe als functie van de invoergrootte?
Zoals te zien is in figuur 2, neemt de uitvoeringstijd van de CPU-implementatie toe naarmate de invoergrootte groter wordt. Bij beide GPU-implementaties daalt de uitvoeringstijd daarentegen eerst, om pas later bij grotere invoergroottes opnieuw te stijgen.

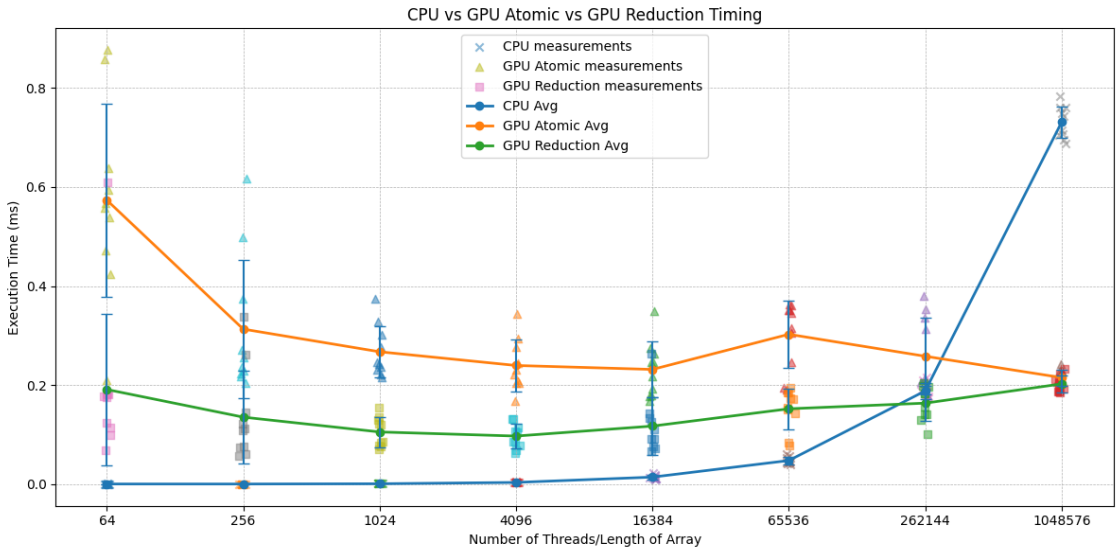
Voor kleine invoergroottes zijn de GPU-implementaties aanzienlijk trager dan de CPU-implementatie. Dit is te wijten aan de relatief grote overhead om de data te kopiëren van het hoofdgeheugen naar het GPU-geheugen. Vanaf een array-lengte van 262144 elementen worden de GPU-implementaties echter sneller dan de CPU-implementatie.

```
N (aantal array elementen/totaal aantal threads): 1048576
# threads per block: 1024
# blocks: 1024
Max is 1048575
----------
Max (CPU): 1048575
Correct? True
Time taken: 0.761263 ms
----------
Max (GPU atomic): 1048575
Correct? True
Time taken: 0.229376 ms
----------
Max (GPU reduction): 1048575
Correct? True
Time taken: 0.187328 ms
```

Figuur 1: Terminal output Labo 2

Daarnaast is de atomic-implementatie trager en vertoont ze meer inconsistentie dan de reductie-implementatie.

De grafiek (Figuur 2) werd opgesteld door voor elke aantal threads het gemiddelde te nemen van tien metingen van de uitvoeringstijd.



Figuur 2: Vergelijking van uitvoeringstijd tussen de CPU-implementatie en GPU-implementaties

4 Vragen

Wat gebeurt er als de invoer groter is dan 2048 (hoe zou u een correcte berekening garanderen en zijn er aanvullende synchronisatiestappen)?

Thread-synchronisatie (`___syncthreads()`) werkt uitsluitend binnen één block. Wanneer de invoer groter is dan 2048 elementen (en dus meerdere blocks vereist), kan er geen synchronisatie

tussen blocks plaatsvinden binnen één kernel-lancering. Om een correcte berekening te garanderen, kan men per block een partiële reductie uitvoeren in shared memory. Elke block berekent hierbij eerst een lokaal maximum, die dan vervolgens ofwel met atomaire operaties gecombineerd kunnen worden in globaal geheugen, ofwel verder gereduceerd worden in een tweede kernel-lancering. Wij hebben ervoor gekozen om het probleem op te lossen met het gebruik van shared memory, om per block het maximum te bepalen. Via een tweede kernel lancering bepalen we het globale maximum [3].

Overweeg om de kernel aan te roepen met veel minder threads dan er gegevenselementen zijn. Bijvoorbeeld een tiende. Hoe zou u ervoor zorgen dat alle elementen worden verwerkt?
Als de kernel met minder threads dan elementen wordt aangeroepen, kan elke thread meerdere elementen verwerken via *Grid-Stride Loops*. Hierbij start elke thread bij zijn globale index en verwerkt vervolgens bijkomende elementen met stappen van `blockDim.x * gridDim.x`. Op deze manier worden alle elementen van de array gegarandeerd verwerkt, ongeacht het aantal threads dat wordt gebruikt [2].

Hoe zou je een 2D-array aanpakken?
Voor de CPU-implementatie wordt de enkele iteratie uitgebreid naar een geneste for-lus over de rijen en kolommen.
In het geheugen wordt een 2D-array opgeslagen in het geheugen als een aaneengesloten 1D-array, waarbij de juiste positie berekend wordt via de indexering $y * breedte + x$. Voor de GPU-implementaties (zowel met atomaire operaties als met reductie) maken we gebruik van een 2D-threadstructuur via `dim3`. Hierdoor kunnen threads efficiënt over de matrix worden verdeeld, waarbij elke thread één of meerdere elementen verwerkt.

Om de uitvoeringstijd van de GPU-reductie verder te minimaliseren is het essentieel om gebruik te maken van het snelle gedeelde geheugen (`__shared__`). Threads binnen een blok kunnen daarmee eerst een lokaal maximum bepalen voordat ze het globale geheugen updaten (wat performanter is) [1].

5 Conclusie

In dit labo werd het maximum van een 1D-array bepaald via een CPU-implementatie en twee GPU-implementaties (atomaire operaties en reductie). De CPU-implementatie vertoont een lineaire toename in uitvoeringstijd naarmate de invoergrootte stijgt. De GPU-implementaties hebben aanvankelijk meer overhead, maar worden vanaf grotere arraylengtes efficiënter dan de CPU.

Dit labo toont aan dat GPU-parallelisatie vooral voordelen biedt bij grote datasets en dat een correcte en efficiënte synchronisatiestrategie essentieel is voor performante resultaten.

Referenties

[1] Cuda in two-dimension — gpu programming.

[2] M. Harris and M. Harris. Cuda pro tip: Write flexible kernels with grid-stride loops. *NVIDIA Technical Blog*, Mar. 2025.

[3] NVIDIA. *CUDA C++ Programming Guide*.

```cpp
#include <iostream>
#include <cuda_runtime.h>
#include <chrono>

// CUDA kernels
__global__ void max_atomic(int *a, int n, int* result) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < n) {
        atomicMax(result, a[idx]);
    }
}

// max reduction voor in 1 block
__global__ void max_reduction(int *a, int n, int* result) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < n) {
        //per 2 elementen vergelijken en maximum opslaan in onderste index van de 2 elementen
        for (int step = 1; step < n; step *= 2) {
            if (idx % (2 * step) == 0) { //iteratie 1: idx 0, 2, 4, ... 2: idx 0, 4, 8, ...
                int other_idx = idx + step; //iteratie 1: idx 1, 3, 5, ... 2: idx 2, 6, 10, ...
                if (other_idx < n) {
                    if (a[idx] < a[other_idx]) {
                        a[idx] = a[other_idx];
                    }
                }
            }
            __syncthreads(); // race conditities vermijden, synchroniseert enkel 1 block
        }

        *result = a[0];
    }
}

__global__ void max_reduction_blocks(int *a, int n, int* block_max) {
    //nu wel gebruik maken van shared memory
    __shared__ int sdata[1024];

    int tid = threadIdx.x;
    int idx = threadIdx.x + blockIdx.x * blockDim.x;

    //load global memory in shared memory
    if (idx < n) {
        sdata[tid] = a[idx];
    } else {
        sdata[tid] = INT_MIN;
    }
```

```
47
48      __syncthreads();
49
50      for (int step = 1; step < blockDim.x; step *= 2) {
51          int i = 2 * step * tid;
52          int other_i = i + step;
53          if (i + step < blockDim.x) {
54              if (sdata[i] < sdata[other_i]) {
55                  sdata[i] = sdata[other_i];
56              }
57          }
58
59          __syncthreads();
60      }
61
62      if (tid == 0) {
63          block_max[blockIdx.x] = sdata[0];
64      }
65
66  }
67
68
69  int main() {
70      const int N = 1048576; //2 ** 20 = 1048576
71      const int threadsPerBlock = 1024;
72      // verdelen in blocks
73      int blocks = (N + threadsPerBlock-1) / threadsPerBlock;
74
75      std::cout << "N (aantal array elementen/totaal aantal threads): " << N <<
        std::endl;
76      std::cout << "# threads per block: " << threadsPerBlock << std::endl;
77      std::cout << "# blocks: "<< blocks << std::endl;
78
79      int a[N];
80
81      for (int i = 0; i < N; i++) {
82          a[i] = i;
83      }
84      // max is always N-1
85
86      std::cout << "Max is " << N-1 << std::endl;
87      std::cout << "----------" << std::endl;
88      //-----Method 1: for loop on CPU-----
89      int* max_cpu = 0;
90      const auto start_cpu = std::chrono::steady_clock::now();
91      for (int i = 1; i < N; i++) {
92          if (a[i-1] > a[i]) {
93              max_cpu = &a[i-1];
94          } else {
95              max_cpu = &a[i];
96          }
```

```
97      }
98      const auto end_cpu = std::chrono::steady_clock::now();
99      const std::chrono::duration<double, std::milli> elapsed_cpu{end_cpu -
        start_cpu};
100     std::string correct_CPU = "False";
101     if (*max_cpu == N-1) correct_CPU = "True";
102
103     std::cout << "Max (CPU): " << *max_cpu << std::endl;
104     std::cout << "Correct? " << correct_CPU << std::endl;
105     std::cout << "Time taken: " << elapsed_cpu.count() << " ms" << std::endl;
106
107     std::cout << "----------" << std::endl;
108     //-----Method 2: GPU kernel using atomic operations-----
109     int *d_a;
110     int* max_atomic_value = 0;
111
112     // Allocate device memory
113     cudaMalloc((void**)&d_a, N * sizeof(int));
114     cudaMalloc((void**)&max_atomic_value, sizeof(int));
115
116     // Copy data from host to device
117     cudaMemcpy(d_a, a, N * sizeof(int), cudaMemcpyHostToDevice);
118     cudaMemcpy(max_atomic_value, &a[0], sizeof(int), cudaMemcpyHostToDevice);
119
120     // Timing
121     cudaEvent_t start_atomic, stop_atomic;
122     cudaEventCreate(&start_atomic);
123     cudaEventCreate(&stop_atomic);
124     cudaEventRecord(start_atomic);
125
126     // Kernel
127     max_atomic<<<blocks, threadsPerBlock>>>(d_a, N, max_atomic_value);
128
129     cudaDeviceSynchronize();
130     cudaEventRecord(stop_atomic);
131     cudaEventSynchronize(stop_atomic);
132     float ms;
133     cudaEventElapsedTime(&ms, start_atomic, stop_atomic);
134
135     int max_val;
136     cudaMemcpy(&max_val, max_atomic_value, sizeof(int), cudaMemcpyDeviceToHost)
        ;
137
138     std::string correct_atomic = "False";
139     if (max_val == N-1) correct_atomic = "True";
140
141     std::cout << "Max (GPU atomic): " << max_val << std::endl;
142     std::cout << "Correct? " << correct_atomic << std::endl;
143     std::cout << "Time taken: " << ms << " ms" << std::endl;
144
145     std::cout << "----------" << std::endl;
```

```
146     //-----Method 3: GPU kernel using reduction-----
147 
148     // single block:
149     /*
150     int* max_reduction_value = 0;
151 
152     // Allocate device memory
153     cudaMalloc((void**)&max_reduction_value, sizeof(int));
154 
155     // Copy data from host to device
156     cudaMemcpy(max_reduction_value, &a[0], sizeof(int), cudaMemcpyHostToDevice)
        ;
157 
158     // Timing
159     cudaEvent_t start_reduction, stop_reduction;
160     cudaEventCreate(&start_reduction);
161     cudaEventCreate(&stop_reduction);
162     cudaEventRecord(start_reduction);
163 
164     // Kernel
165     max_reduction<<<blocks, threadsPerBlock>>>(d_a, N, max_reduction_value);
166 
167     // stop timing
168     cudaDeviceSynchronize();
169     cudaEventRecord(stop_reduction);
170     cudaEventSynchronize(stop_reduction);
171     float ms2;
172     cudaEventElapsedTime(&ms2, start_reduction, stop_reduction);
173     */
174     // multiple blocks:
175     int *d_input = d_a;
176     int *d_output;
177     int current_n = N;
178 
179     //Timing
180     cudaEvent_t start_reduction, stop_reduction;
181     cudaEventCreate(&start_reduction);
182     cudaEventCreate(&stop_reduction);
183     cudaEventRecord(start_reduction);
184 
185     while (current_n > 1) {
186         int blocks = (current_n + threadsPerBlock -1) / threadsPerBlock;
187 
188         cudaMalloc(&d_output, blocks * sizeof(int));
189 
190         max_reduction_blocks<<<blocks, threadsPerBlock>>>(d_input, current_n,
        d_output);
191 
192         cudaDeviceSynchronize();
193 
194         if (d_input != d_a) {
```

```
            cudaFree(d_input);
        }

        d_input = d_output;
        current_n = blocks;
    }

     // stop timing
    cudaDeviceSynchronize();
    cudaEventRecord(stop_reduction);
    cudaEventSynchronize(stop_reduction);
    float ms2;
    cudaEventElapsedTime(&ms2, start_reduction, stop_reduction);

    int max_val_reduction;
    cudaMemcpy(&max_val_reduction, d_input, sizeof(int), cudaMemcpyDeviceToHost
    );

    std::string correct_reduction = "False";
    if (max_val_reduction == N-1) correct_reduction = "True";

    std::cout << "Max (GPU reduction): " << max_val_reduction << std::endl;
    std::cout << "Correct? " << correct_reduction << std::endl;
    std::cout << "Time taken: " << ms2 << " ms" << std::endl;

    // Free device memory
    cudaFree(d_a);
    cudaFree(d_input);
    cudaFree(max_atomic_value);


    return 0;
}
```

Codefragment 1: Code labo 2

FACULTEIT INDUSTRIELE INGENIEURSWETENSCHAPPEN
TECHNOLOGIECAMPUS GENT
Gebroeders De Smetstraat 1
9000 GENT, België
tel. + 32 92 65 86 10
fax + 32 92 25 62 69
iiw.gent@kuleuven.be
www.iiw.kuleuven.be

LID VAN ASSOCIATIE KU LEUVEN